Computers & Operations Research 136 (2021) 105454

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor

A dynamic and probabilistic orienteering problem

Enrico Angelelli [a], Claudia Archetti [b], Carlo Filippi [a,*], Michele Vindigni [a]

[a] Department of Economics and Management, University of Brescia, Contrada S. Chiara 50, 25122 Brescia, Italy
[b] Department of Information Systems, Decision Sciences and Statistics, ESSEC Business School in Paris, 3 Avenue Bernard Hirsch, 95000 Cergy, France

ARTICLE INFO

Keywords:
Routing
Heuristics
Random requests
Dynamic vehicle routing
Orienteering problem

ABSTRACT

We consider an online version of the orienteering problem, where stochastic service requests arise during a first time interval from customers located on the nodes of a graph. Every request must be accepted/rejected in real time. Later, a vehicle must visit the accepted customers during a second time interval. Each accepted request implies a prize, depending on the customer, and a service cost, depending on the routing decisions. Moreover, an accepted request implies a reduction of the routing time available for possible future requests. Each acceptance/rejection decision is made to maximize the expected profit, i.e., the difference between expected prices and expected service costs.

We formulate the problem as a Markov Decision Process and derive analytical expressions for the transition probabilities and the optimal policy. Since an exact policy computation is intractable, we design and test several heuristic approaches, including static approximation, simple greedy (non-anticipatory) methods, Sample Average Approximation (SAA) of the objective function using Monte Carlo sampling of future events. We perform extensive computational tests on the proposed algorithms and discuss the pros and cons of the different methods.

1. Introduction

Dynamic and stochastic routing problems have received increasing attention in recent years. On the academic side, efficient algorithmic approaches and increased computational power allow to face the complexity of stochastic models (Psaraftis et al., 2016); on the practical side, e-commerce and technological advances require flexible and real-time decision making (Ulmer et al., 2018). Moreover, time-constrained deliveries have been one of the fastest growing segments of the delivery business (Campbell and Thomas, 2008), and it is argued that selective vehicle routing problems are more appropriate than the conventional routing problems in handling uncertainty with limited resources (Allahviranloo et al., 2014).

We study a time-constrained, selective, dynamic and stochastic routing problem. Consider a transportation company that collects and manages requests for long-haul transportation from numerous customers. Usually, shipments from a certain geographical zone are collected by a vehicle and delivered to a carrier's facility. Here, shipments are consolidated according to their destination and sent to other facilities by long-haul transportation (Pillac et al., 2013). In particular, we consider the manager of a specific facility who has to organize the picking-up of goods for the next-day overnight shipments. At the beginning of the day, the manager has a portfolio of *mandatory requests* and can figure out a portfolio of *spot requests*. Mandatory requests arise from customers with a consolidated relationship of mutual trust with the carrier and use the carrier as a preferential one. These customers typically have a contract for a continuous and regular service. It is understood that all these requests are acquired, constitute an obligation for the carrier and provide a certain revenue. Spot requests arise dynamically on an e-market place from occasional customers. A spot request may be accepted or rejected, but the decision must be taken in real time, or after a short negotiation. If a spot request is accepted, then it becomes binding on both parties.

We consider a decision problem encompassing two successive days. During the first day, the manager receives spot requests and, for each request, he/she has the opportunity to either accept or reject the corresponding shipment. During the second day, the customers associated with mandatory requests and accepted spot requests must be visited by a vehicle to pick-up goods. An accepted request implies a *prize*, i.e., the revenue obtained from the shipment service, and an *operational cost*, i.e., the cost of the deviation in vehicle's route to reach the customer and picking-up the goods. Moreover, accepting a request reduces the

* Corresponding author.
E-mail addresses: enrico.angelelli@unibs.it (E. Angelelli), archetti@essec.edu (C. Archetti), carlo.filippi@unibs.it (C. Filippi), michele.vindigni@unibs.it (M. Vindigni).

https://doi.org/10.1016/j.cor.2021.105454
Received 16 July 2020; Received in revised form 11 May 2021; Accepted 28 June 2021
Available online 9 July 2021

vehicle's *time budget* (Ulmer et al., 2018), and thus may prevent the acceptance of future and maybe more profitable requests. Indeed, if a vehicle returns to the facility too late on the second day, it may delay all the scheduled overnight shipments. Consequently, any vehicle must return to the facility within a sharp deadline.

Our aim is to design and implement a service policy that helps the manager to decide about the acceptance or rejection of a spot request according to the criterion of maximum expected profit, where the profit is the algebraic sum of prize and operational cost. Notice that the acceptance/rejection decision is constrained by and affects routing decisions that will be implemented at a later time. An integrated decision process, possibly anticipatory of future events, is then required.

We assume that the manager has a single vehicle for the picking-up service, and that the spatial location of both mandatory and spot requests is known in advance. Thus, we model the problem on a complete directed graph where nodes correspond to requests. There are two types of nodes: mandatory and spot. Mandatory nodes must be visited, spot nodes may be visited if they originate a request. A distinguished feature of our problem is that the decision process about acceptance/rejection of spot requests takes place before the actual routing is fulfilled. We thus work with two successive time intervals: a decision time interval $(0, T)$, placed in day 0, and a routing time interval $(0, D_{\max})$, placed in day 1, see Fig. 1. During the decision time interval, spot requests arise randomly according to a known, time-dependent, probability law and the decision maker must accept/reject them in real time; during the routing time interval, a vehicle starts from the facility, visits the mandatory nodes and the spot nodes that have been accepted, collects the parcels, and return to the facility within the time limit. We assume that the traveling and service times are deterministic. Moreover, we assume that the quantity of goods associated with each request is comparatively small with respect to vehicle capacity, so that the vehicle may be regarded as uncapacitated. Note that this latter assumption is consistent with many practical settings. Indeed, last-mile pickup and delivery of light parcels is characterized by tightness of customer service times or maximum working time for drivers, while capacity is rarely binding.

In summary, we study a dynamic and stochastic extension of the Orienteering Problem (OP) (Gunawan et al., 2016), where requests for service arise randomly one at a time along a given decision time interval according to a known probability law, and the acceptance/rejection of every single request must be made in real time. The objective is the maximization of the expected profit, where the profit is given by the difference between the expected total prize (including possible future requests) and the expected traveling cost (including the visits to possible future requests).

The main contributions of this paper can be summarized as follows:

- We define the Dynamic and Probabilistic Orienteering Problem (DPOP), that generalizes the well-known OP by introducing dynamic acceptance/rejection of probabilistic customers. Differently from most dynamic routing problems considered in the literature, in DPOP online decisions have to be taken before the vehicle leaves the depot, so that the whole route can be revised after each acceptance/rejection decision. The objective to be maximized is the expected profit.
- We formalize the problem as a Markov Decision Process (MDP), formally characterizing the transition probabilities and the optimal decision policy.
- We develop several heuristic approaches, including static approximation, simple greedy (non-anticipatory) methods, Sample Average Approximation (SAA) of the objective function using Monte Carlo sampling of future events.
- We perform extensive computational tests on the proposed algorithms and discuss the pros and cons of the different methods on the specific problem.

The paper is organized as follows. After revising the most relevant literature in Section 2, we describe the decision problem as a MDP in Section 3. In Section 4, the transition probabilities are derived and an analytic recursive formula for the evaluation of the optimal policy is obtained. In Section 5, we propose several approximated algorithmic approaches, whose computational performance is evaluated and discussed in Section 6. Finally, some conclusions are drawn in Section 7.

2. Literature review

The DPOP is a dynamic and stochastic vehicle routing problem (DSVRP) which generalizes the Orienteering Problem (OP). Specifically, the stochastic feature refers to the presence of random requests from a set of potential customers, while the dynamic feature refers to the fact that random requests reveal over time and need immediate response. Moreover, a specific feature of the DPOP is that every acceptance/rejection decision is taken before the vehicle actually starts the tour. As a consequence, the order of visit of the accepted customers can be modified thoroughly after any acceptance decision. On one hand, this feature makes the problem very different from most dynamic routing problems, where requests arise while the vehicle is on the road. On the other hand, the same feature is shared by routing problems related to *attended home delivery* (AHD) services, where, however, the main issue is the management of delivery time windows. In this section, we review the contributions on stochastic and dynamic OPs and on DSVRPs with stochastic requests mostly related to our problem.

Concerning stochastic orienteering problems, the reference closest to the present work is by Angelelli et al. (2017), who study the Probabilistic OP (POP). The problem is formulated on a directed graph, where each

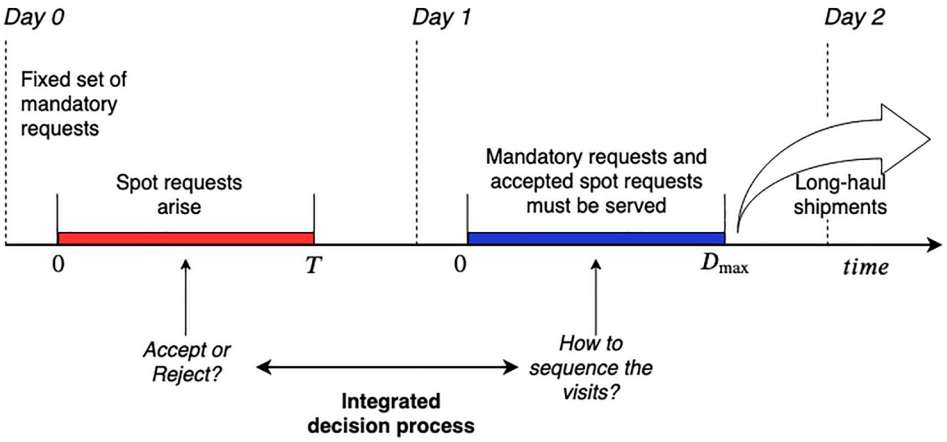

Fig. 1. Graphical illustration of the decision process.

customer is available for visit only with a certain probability. In a first stage, a subset of nodes has to be selected and a corresponding a priori path has to be determined such that the server can visit all customers in the subset and reach the destination without exceeding a time budget. In a second stage, after the list of available customers is revealed, the server follows the a priori path by skipping the absent nodes. The POP consists in determining a first-stage solution that maximizes the expected profit of the second-stage path, where the expected profit is the difference between the expected total prize and the expected total cost. A branch-and-cut exact approach and several matheuristic methods are proposed. In fact, the POP is a static variant of the problem we study in this paper, where the probability of request from a customer is fixed and an a priori route based on all possible requests is evaluated. In the DPOP, the probability of a request is a function of time, and once a request arises, we have to decide immediately whether to accept it or not.

Other stochastic variants of the OP are proposed by İlhan et al. (2008), who discuss the OP with Stochastic Profits (OPSP); Campbell et al. (2011), who study the OP with Stochastic Travel and Service times (OPSTS); Evers et al. (2014), who consider the OP with Stochastic Weights (OPSW), where weights are associated with travel costs, travel time (including service time) or fuel consumption on arcs; Zhang et al. (2014), who study the Stochastic OP with Time Windows (SOPTW), where an uncertain waiting time at a customer results from a possible queue of competitors arrived earlier. Notice that all the OPs mentioned so far are static problems. Zhang et al. (2018) consider a dynamic variant of the SOPTW, where the server decides dynamically the next customer to visit, allowing the visit to a customer previously abandoned. The decision strategy is identified by an approximate dynamic programming approach based on rollout algorithms. We refer to Gunawan et al. (2016) for additional references to stochastic extensions of the OP.

Concerning AHD problems, Campbell and Savelsbergh (2005) are the first to face grocery delivery services by optimization techniques. Their setting is similar to ours: requests may arise from known customers and according to known probabilities, that reduce over time; the objective is the same as the DPOP, i.e., expected profit. However, there are important differences: requests are associated with narrow time windows, there are multiple vehicles and capacity constraints. Concerning the algorithmic approach, Campbell and Savelsbergh (2005) propose an insertion heuristic to determine the feasibility of a request and different strategies to evaluate the expected profit associated with a decision. When a new request occurs at time t, their insertion heuristic considers a static problem where the prize associated with every not-yet-materialized request is adjusted according to its probability of occurrence computed at time t. Hence, Campbell and Savelsbergh (2005) face a more general problem than the DPOP, but they analyze a single algorithmic approach. They consider neither simpler greedy heuristics, nor Sample Average Approximation methods, as we do in our study. Moreover, no theoretical analysis is provided in Campbell and Savelsbergh (2005), whereas we provide an analytical description of the stochastic process embedded in our optimization problem. The successive studies on AHD problems are mainly focused on time interval management and incentive schemes to induce customer to choose the best time window (from a routing viewpoint). See, e.g., Campbell and Savelsbergh (2006) and Agatz et al. (2011).

Azi et al. (2012) consider a dynamic VRP where vehicles perform multiple routes during their workday. Requests arrive according to a Poisson process and must be accepted or rejected in real time. An accepted request is inserted in one of the routes that will be executed later. The authors adapt an ALNS algorithm developed for the static version of the problem to the dynamic version. The expected gain obtained from the acceptance of a new request is evaluated by inserting it on a number of request scenarios, generated at the start of the algorithm. As in the DPOP, every vehicle starts from the depot with a fixed sequence of customers to visit and the objective is the expected profit. Differently from DPOP, the decision interval and the routing interval are overlapping, and multiple vehicles start asynchronously over time.

Ulmer and Thomas (2019) consider the capacitated customer acceptance problem with stochastic requests (CAPSR), which is very similar to DPOP. As in the DPOP, they consider a single vehicle and there are no time windows associated with requests. However, the vehicle is capacitated, the objective is the expected revenue (no costs are considered), and a (fixed) service time is spent at each visited customer. They also assume that requests arise according to a Poisson process and that prize and demand associated with each request are unknown before the request arises. Finally, the CAPSR is not the focus of the paper, primarily devoted to the development of a general-purpose "meso-parametric" value function approximation.

Concerning general DSVRPs, a first distinction depends on whether stochastic information about possible future requests is available and used in the decision process or not. In the former case, we have anticipatory approaches, in the latter, not anticipatory approaches (Ulmer et al., 2018). Notice that the approach used in this paper is anticipatory, since decisions take into account the possible future requests. Anticipatory approaches are usually based on sampling. More specifically, future customer requests are sampled on the demand space or the graph nodes, according to the assumed probability distribution. This allows to better evaluate the impact of current decisions, but requires a significant computational effort. Sampling approaches for DSVRPs with random requests are proposed by Bent and Van Hentenryck (2004), Flatberg et al. (2007) andSungur et al. (2010). Short-term sampling is used by Ghiani et al. (2009) for a dynamic pickup and delivery problem. Sampling based on historical data is used by Hvattum et al. (2006) to solve a real-world case study. Ghiani et al. (2012) propose an anticipatory insertion waiting approach, based on the concept of center of gravity of the potential future requests. Finally, Ulmer et al. (2018) design an anticipatory time budgeting heuristic, drawing on methods of approximate dynamic programming. Notice that, among the cited works, only Ghiani et al. (2012) consider random requests from nodes on a graph as in our work, whereas all other papers consider a probabilistic distribution of requests on a continuous 2-dimensional space. We refer to Ritzinger et al. (2016) and to Psaraftis et al. (2016) for recent surveys on DSVRPs.

3. Problem description

Let $V = \{1, 2, \ldots, n\}$ be the set of n potential request nodes and let $V_M \subseteq V$ be the set of mandatory request nodes. Mandatory requests arise from regular customers and must be satisfied. The remaining potential request nodes $V_S = V \setminus V_M$ correspond to spot requests. We will call them *optional* request nodes from now on. In general, there is no one-to-one correspondence between nodes in V and actual customers. More precisely, a regular customer r (originating a mandatory request) might sometimes place a spot request. In this case, two distinct nodes $u \in V_M$ and $v \in V_S$ would be associated with r.

Consider first a deterministic setting. Assume that we have fixed a subset W of request nodes to serve, $V_M \subseteq W \subseteq V$, with all requests from nodes in W confirmed. Consider a directed graph $G = (\overline{V}, A)$, with node set $\overline{V} = V \cup \{0, n+1\}$, where 0 is the *origin* and $n+1$ is the *destination* (possibly coinciding with the origin), and arc set $A = \{(i,j) : i \in V \cup \{0\}, j \in V \cup \{n+1\}\}$. Let p_i be the prize associated with node $i \in V$, and let $\tau_{i,j}$ be the traveling time associated with arc $(i,j) \in A$. We assume also a traveling cost proportional to the traveling time, so that traveling through arc (i,j) implies a cost $C \cdot \tau_{i,j}$, where C is a properly defined positive constant. A vehicle must start from the origin, visit all nodes in W according to some order, and then end at the destination.

Let $\tau_{\min}(W)$ denote the duration of a minimum cost elementary path from 0 to $n+1$ visiting all nodes in W. Then, set W is *feasible* if and only if $\tau_{\min}(W) \leqslant D_{\max}$. Moreover, $C \cdot \tau_{\min}(W)$ is the minimum cost of the path. In this deterministic setting, the profit associated with W is the difference between the total collected prize and the minimum traveling cost:

$$P(W) = \sum_{i \in W} p_i - C \cdot \tau_{\min}(W).$$

Moving to a stochastic context, we assume that requests are placed by optional customers during a time window $(0, T)$. A random variable T_i, representing the time in which the service request is placed, is associated with every node $i \in V$. For mandatory nodes, the distribution is degenerate: $T_i = 0$ with probability 1 for $i \in V_M$. For all $i \in V_S$, we have:

$$T_i = \begin{cases} \overline{T}_i & \text{with probability } \theta_i, \\ T+1 & \text{with probability } 1-\theta_i, \end{cases}$$

where $\overline{T}_i$ is a continuous random variable with known Cumulative Density Function (CDF), denoted as $\overline{F}_i$, and known Probability Density Function (PDF), denoted as $\overline{f}_i$. Thus, we have

$$\overline{F}_i(t) = \begin{cases} 0 & \text{if } t \leqslant 0, \\ \int_0^t \overline{f}_i(x)dx & \text{if } 0 < t \leqslant T, \\ 1 & \text{if } t > T. \end{cases}$$

For $i \in V_S, \theta_i$ is the probability that a request arises from i during the time horizon $(0, T)$; $T+1$ is a conventional point in time where "late" or "no-show" requests are placed. The CDF of T_i can thus be written as

$$F_i(t) = \theta_i \overline{F}_i(t) + (1-\theta_i) I(t), \tag{1}$$

where $I(t)$ is an indicator function defined as

$$I(t) = \begin{cases} 1 & \text{if } t \geqslant T+1, \\ 0 & \text{otherwise.} \end{cases}$$

We assume that requests arise independently among nodes.

We model the problem as a MDP (see, e.g., Powell et al., 2012 for a definition of MDPs and their components). The components of the MDP proposed for the DPOP are described in the following.

Stages. A stage is a situation where a decision has to be taken. A *proper stage* is the arrival of a request from a node in V_S at time $t \in (0, T)$; the *final stage* is the achievement of the time horizon T. In a proper stage, the decision is about acceptance/rejection of an optional request; in the final stage, the decision is about the visit sequence of the mandatory and accepted optional requests. A proper stage is identified by an ordered pair (t, i), where t is the time of occurrence and i is the node asking for service; the final stage is identified by the ordered pair $(T, \varnothing)$. Since every request node may originates at most one request, we have at most $|V_S|$ proper stages. Let $k \in \{1, 2, \ldots, h\}$ be the stage counter, where $h \leqslant |V_S| + 1$.

States. A state collects all the information necessary to describe the system at any stage. To define a state, notice that at any time $t \in (0, T)$ we have a set $R(t)$ of nodes whose requests have been *received*, $V_M \subseteqq R(t) \subseteqq V$, and a set of nodes $W(t)$ whose requests have been *received and accepted*, $V_M \subseteqq W(t) \subseteqq R(t)$. Moreover, we assume that $\tau_{\min}(W(t)) \leqslant D_{\max}$. Thus, we describe the state of the system at the kth stage as the ordered pair (R^k, W^k), where R^k is the set of nodes that have already originated a request, W^k is the set of nodes whose request has been accepted. The initial state is $(R^1, W^1) = (V_M, V_M)$.

Actions. An action occurs at every stage, where a decision has to be taken and implemented. A *policy* π is a rule to choose the proper action. We denote by $\pi(R, W, t, i) \in \{\varnothing, \{i\}\}$ the action chosen by policy π when a request (t, i) arrives with the system at state (R, W). In the final stage, the action $\pi(R^h, W^h, T, \varnothing)$ is the computation of $\tau_{\min}(W^h)$ and a corresponding sequence of visit of the nodes in W^h.

Transitions. Once an action is chosen in a proper stage, we have a state transition. The transition associated with the kth proper stage, identified by the request (t, i), is defined as follows:

$$(R^k, W^k) \longrightarrow (R^{k+1} = R^k \cup \{i\}, W^{k+1} = W^k \cup \pi(R^k, W^k, t, i)).$$

Objective. The objective of the problem at any stage is the maximization of the expected profit. More precisely, let $EP_\pi(R, W, t, i)$ denote the expected profit produced by policy π when the proper stage (t, i) occurs with system at state (R, W). Let Π denote the set of all policies. Then, we are interested in a policy $\pi^\star \in \Pi$ such that $EP_{\pi^\star}(R, W, t, i) \geqslant EP_\pi(R, W, t, i)$ for all $\pi \in \Pi$, state (R, W), and proper stage (t, i).

4. Optimal policy

In this section, we define an optimal policy for the problem described above by characterizing recursively its expected profit function.

At the final stage, the profit associated with any policy π is deterministic and is computed as

$$EP_\pi(R^h, W^h, T, \varnothing) = \sum_{i \in W^h} p_i - C \cdot \tau_{\min}(W^h). \tag{2}$$

For proper stages, to compute the expected profit function we need the transition probabilities to the next stage. Assume that the proper stage (t^{k+1}, i^{k+1}) occurs with system at state (R^k, W^k). At this point in time, the next stage will be triggered by either a new request or the end of the time horizon. Hence, the next stage will occur at time

$$t_{\min}(t^{k+1}, R^{k+1}) = \min\{\min\{T_j : j \in V \backslash R^{k+1}, T_j > t^{k+1}\}; T\}$$

where $R^{k+1} = R^k \cup \{i^{k+1}\}$. If $t_{\min}(t^{k+1}, R^{k+1}) = T$, then the next stage is the final stage, and formula (2) is used to evaluate the expected profit associated with the acceptance or rejection of i^{k+1}. If $t_{\min}(t^{k+1}, R^{k+1}) < T$, then the next stage is a proper stage. In this case, to evaluate the expected profit associated with the acceptance or rejection of i^{k+1}, we need to characterize the probability distribution of the next stage conditioned to the current time and system state. This is done in the following theorem, whose proof is provided in Appendix A.

Theorem 1. *Assume that the proper stage (t^{k+1}, i^{k+1}) occurs with system at state (R^k, W^k). Then, the probability that the next stage is a proper stage and involves node $i \in V \backslash R^{k+1}$ is*

$$p_{\min}(i | t^{k+1}, R^{k+1}) = \int_{t^{k+1}}^{T} \left[\prod_{j \in V \backslash (R^{k+1} \cup \{i\})} \left(\frac{1-\theta_j \overline{F}_j(t)}{1-\theta_j \overline{F}_j(t^{k+1})} \right) \right] \frac{\theta_i \overline{f}_i(t)}{1-\theta_j \overline{F}_i(t^{k+1})} dt \tag{3}$$

Moreover, the probability that the next stage is the final stage is

$$p_{\min}(\varnothing | t^{k+1}, R^{k+1}) = \prod_{j \in V \backslash R^{k+1}} \left(\frac{1-\theta_j}{1-\theta_j \overline{F}_j(t^{k+1})} \right). \tag{4}$$

To simplify notation, for all $i \in V \backslash R^{k+1}$, let us denote by $\phi_{i | t^{k+1}, R^{k+1}}(t)$ the function integrated in (3), i.e.,

$$\phi_{i | t^{k+1}, R^{k+1}}(t) = \left[\prod_{j \in V \backslash (R^{k+1} \cup \{i\})} \left(\frac{1-\theta_j \overline{F}_j(t)}{1-\theta_j \overline{F}_j(t^{k+1})} \right) \right] \frac{\theta_i \overline{f}_i(t)}{1-\theta_j \overline{F}_i(t^{k+1})}.$$

Now, the expected profit function of the optimal policy $\pi^\star$ can be expressed recursively as follows:

$$\begin{aligned} EP_{\pi^\star}(t^k, R^k, W^k, t^{k+1}, i^{k+1}) = \max\{ & \\ \sum_{i \in V \backslash R^{k+1}} \int_{t^{k+1}}^{T} & EP_{\pi^\star}(t^{k+1}, R^{k+1}, W^k \cup \{i^{k+1}\}, t, i) \phi_{i | t^{k+1}, R^k}(t) dt \\ + EP_{\pi^\star}(t^{k+1}, R^{k+1}, W^k \cup \{i^{k+1}\}, T, \varnothing) & p_{\min}(\varnothing | t^{k+1}, R^{k+1}); \\ \sum_{i \in V \backslash R^{k+1}} \int_{t^{k+1}}^{T} & EP_{\pi^\star}(t^{k+1}, R^{k+1}, W^k, t, i) \ \phi_{i | t^{k+1}, R^k}(t) dt \\ + EP_{\pi^\star}(t^{k+1}, R^{k+1}, W^k, T, \varnothing) & p_{\min}(\varnothing | t^{k+1}, R^{k+1}) \} \end{aligned}$$

where the request from i^{k+1} is accepted or not according to the maximization of the expected value of the profit function.

Except for trivial cases, an exact evaluation of the above formula is intractable. Hence, we have to consider and evaluate suboptimal policies.

5. Solution approaches

Each time an event occurs, we should maximize the recursive expected profit function $EP_{\pi^\star}$. As evaluating this function is in general computationally prohibitive, we have to rely on approximations.

In general, one may think about two classes of heuristic approaches:

- *Static approaches*. The idea in this case is to develop a solution approach that is applied only once, at time 0, with the information available at time 0. This means that the solution will determine a priori which are the requests for which the service will be provided if requested and those for which the service will not be provided in any case. Then, the final solution is the one where the requests served are those that have actually arisen among the selected ones.
- *Dynamic approaches*. The idea in this case is to reoptimize the problem at each proper stage, i.e., whatever is the decision rule, it is applied each time a new request is placed taking into account what are the requests for which a decision has already been taken in previous stages and what is the information on remaining requests at the time the reoptimization is run.

5.1. Static approach

We propose one algorithm that exploits the information available at time 0, solves a problem, and use the corresponding solution to take future decisions. Notice that this approach does not take into account the dynamic nature of the DPOP. In particular, the problem solved at time 0 corresponds to the Probabilistic Orienteering Problem (POP) studied in Angelelli et al. (2017). In fact, the POP can be seen as the 'static' version of the DPOP where each customer is associated with a probability of placing a request which does not depend on time and the decision is taken at time 0 and never modified. The objective is to determine a route that does not exceed $D_{\max}$ and maximizes the expected value of the difference between the collected profit and the traveling cost.

The POP is an extremely difficult problem, as shown in Angelelli et al. (2017), for which only small instances can be solved to optimality, Thus, we opted for a heuristic solution for the POP which works as follows. For each customer i, we multiply the prize p_i by θ_i. This way, we obtain a deterministic problem corresponding to a variant of the classical OP where a subset of the customers must be visited. This problem is solved by adapting the heuristic algorithm presented in Chao et al. (1996) for the OP, where the visit to customers in V_M is imposed and the duration of the route does not exceed D_{max}.

The solution determines the subset of requests for which the service will be provided in case they appear. If a request that has not been selected by the POP solution appears, then the service will be denied. In the following, we call this algorithm `OP-One-Shot`.

5.2. Dynamic approaches

We propose two types of dynamic approaches for the solution of the DPOP. The idea behind each of the approach is to recompute an approximated value of $EP_{\pi^\star}$ at each proper stage. The goal is to determine whether to accept or not the new request. Notice that, considering time as a continuous variable, the occurrence of two simultaneous requests has probability zero. Thus, a common feature of the two approaches is that they take a decision on one request at a time. Another common feature is that, before any decision, the probabilities of the future possible optional requests are recomputed as follows. Recall that, when the $(k+1)$th request arises at time t^{k+1} from node i^{k+1}, we have a set R^k of requests already received and a set $W^k \subseteq R^k$ of requests accepted. In addition, $R^{k+1} = R^k \cup \{i^{k+1}\}$. The probability θ_i^{k+1} that a request will arise from a node $i \in V \backslash R^{k+1}$ can be computed using the CDF $F_{i|t^{k+1}}$ of the random variable T_i, given that no request has occurred from i in $[0, t^{k+1}]$. For all $t \in (t^{k+1}, T)$, using (1) we have:

$$F_{i|t^{k+1}}(t) = \frac{\mathbb{P}(t^{k+1} < T_i \leq t)}{\mathbb{P}(T_i > t^{k+1})} = \frac{F_i(t) - F_i(t^{k+1})}{1 - F_i(t^{k+1})} = \frac{\theta_i(\overline{F}_i(t) - \overline{F}_i(t^{k+1}))}{1 - \theta_i \overline{F}_i(t^{k+1})}. \quad (5)$$

Thus,

$$\theta_i^{k+1} = F_{i|t^{k+1}}(T) = \frac{\theta_i(1 - \overline{F}_i(t^{k+1}))}{1 - \theta_i \overline{F}_i(t^{k+1})}. \quad (6)$$

The two dynamic approaches are the following:

1. *Myopic approaches*. These approaches take a decision by analyzing the information available at the time the decision is taken.
2. *Monte Carlo (MC) approaches*. These approaches use MC simulation to predict the policy expected performance over accept/reject decision. In particular, after arrival of request i, a number of scenarios about future possible request arrivals are generated. For each of them the value obtained by a policy is evaluated under both rejection and acceptance of request i.

We now detail the different approaches belonging to the two classes above.

5.2.1. Myopic approaches

A myopic approach takes a decision on the proper stage (t^{k+1}, i^{k+1}) on the basis of the information available at time t^{k+1}.

OP-Multi-Shot. `OP-Multi-Shot` consists in solving a POP at each proper stage. More precisely, when the stage (t^{k+1}, i^{k+1}) occurs, we solve a POP instance where: (a) the nodes in W^k are mandatory; (b) node i^{k+1} is optional but has probability 1; (c) nodes $i \in V \backslash R^{k+1}$ are optional with probability θ_i^{k+1}, see (6). We use the same solution algorithm used in `OP-One-Shot`. Request from i^{k+1} is accepted if and only if it is served in the POP solution.

Greedy algorithms. Greedy algorithms take a decision on the basis of simple rules. We defined four rules, which generated four greedy algorithms. For any path $\mathscr{P}$ from origin 0 to destination $n+1$, let $\pi(\mathscr{P})$ be the duration of $\mathscr{P}$, and let $\Delta_{i,\mathscr{P}}$ be the cheapest insertion time of inserting request node i in path $\mathscr{P}$. Let $\mathscr{P}^k$ be the path built at time t^k to visit request nodes in W^k. Initially, $\mathscr{P}^0$ is the minimum cost elementary path from 0 to $n+1$ visiting all nodes in $W^0 = V_M$. Then, the four greedy algorithms are the following:

1. `Profitable Greedy`: Request i^{k+1} is accepted if $p_{i^{k+1}} - C \cdot \Delta_{i^{k+1},\mathscr{P}^k} \geq 0$ and $\pi(\mathscr{P}^k) + \Delta_{i^{k+1},\mathscr{P}^k} \leq D_{max}$. The idea is that the request of customer i^{k+1} is accepted only if it provides a positive net benefit on the current solution represented by $\mathscr{P}^k$. This rule can be seen as a 'conservative' rule as the acceptance is limited to requests that provide an 'immediate' net benefit, i.e., a benefit that does not depend on what may happen in the future.
2. `Feasible Greedy`: Request i^{k+1} is accepted if $\pi(\mathscr{P}^k) + \Delta_{i^{k+1},\mathscr{P}^k} \leq D_{max}$. This rule may be seen as an 'optimistic' rule as it accepts all requests that can be feasibly satisfied, even in the case they do not provide an immediate net benefit, in the hope of advantageously combine them with future accepted requests.
3. `Profitable Look-ahead Greedy`: This rule uses some heuristic evaluations on the expected effects of accepting or rejecting the request from node i^{k+1}. The basic idea is to accept a request not immediately profitable if it might produce synergistic effects with

other requests that have not yet materialized. To explain how the rule works, we need some additional notation.

- Let $\mathscr{P}^{k+1}$ be the tour obtained from $\mathscr{P}^k$ when request i^{k+1} is inserted through cheapest insertion. Notice that $\pi(\mathscr{P}^{k+1}) = \pi(\mathscr{P}^k) + \Delta_{i^{k+1},\mathscr{P}^k}$. In the following definitions, $\mathscr{P}$ can be either $\mathscr{P}^k$ or $\mathscr{P}^{k+1}$.
- $I(\mathscr{P}) = \{i \in V \backslash R^{k+1} : p_i - C{\cdot}\Delta_{i,\mathscr{P}} \geqslant 0\}$ is the set of 'interesting' possible future requests, i.e., requests that would provide an increase in the objective function if they were accepted and inserted in path $\mathscr{P}$ by cheapest insertion.
- $AEP(I(\mathscr{P})) = \frac{1}{|I(\mathscr{P})|}\left(\sum_{i \in I(\mathscr{P})} \theta_i^{k+1}(p_i - C{\cdot}\Delta_{i,\mathscr{P}})\right)$ is an estimated expected profit from a request in $I(\mathscr{P})$.
- $N_{call}(I(\mathscr{P})) = \sum_{i \in I(\mathscr{P})} \theta_i^{k+1}$ is an estimate of the number of customers in $I(\mathscr{P})$ that will place a request.
- $\tau(V)$ is the duration of the path from 0 to $n+1$ visiting all nodes in V.
- $UC = \frac{\tau_{\min}(V)}{|V|}$ is the estimated time needed to visit one request in V.
- $N_{serve}(I(\mathscr{P})) = \frac{D_{\max} - \pi(\mathscr{P})}{UC}$ is an estimate of the number of requests in $I(\mathscr{P})$ that may be served given the residual time availability.
- $P(I(\mathscr{P})) = AEP(I(\mathscr{P})){\cdot}\min\{N_{call}(I(\mathscr{P})), N_{serve}(I(\mathscr{P}))\}$ is an estimate of the profit that may be obtained from requests in $I(\mathscr{P})$.

The `Profitable Look-ahead Greedy` works as follows:

(a) If $\pi(\mathscr{P}^k) + \Delta_{i^{k+1},\mathscr{P}^k} > D_{max}$ then discard i^{k+1} and stop.
(b) If $p_{i^{k+1}} - C{\cdot}\Delta_{i^{k+1},\mathscr{P}^k} \geqslant 0$ then accept i^{k+1} and stop.
(c) If $p_{i^{k+1}} - C{\cdot}\Delta_{i^{k+1},\mathscr{P}^k} + P(I(\mathscr{P}^{k+1})) \geqslant P(I(\mathscr{P}^k))$ then accept i^{k+1}, else discard it.

4. `Feasible Look-ahead Greedy`: This greedy heuristic works as the `Profitable Look-ahead Greedy` except for point (b) which is ignored. The rationale is to give more emphasis to future synergistic effects than to current local positive effects.

5.2.2. Monte Carlo (MC) approaches

At each proper stage associated with request (t^{k+1}, i^{k+1}), MC approaches generate scenarios for requests that may still appear. We present two approaches. Both of them are based on the following four phases:

1. *Feasibility check*: If $\pi(\mathscr{P}^k) + \Delta_{i^{k+1},\mathscr{P}^k} > D_{max}$, then i^{k+1} is rejected and the procedure stops.
2. *Monte Carlo time sampling*: It randomly generates a set $\mathscr{S}$ of scenarios where, for each request in $V \backslash R^{k+1}$, the time at which the request arises is determined. More precisely, each scenario s is a sequence of stages $(t_{i,s}, i)$ where $i \in V \backslash R^{k+1}$ and $t^{k+1} < t_{i,s} < T$. Note that an arrival time is generated for every $i \in V \backslash R^{k+1}$, but only requests with arrival time less than T are kept in scenario s.
3. *Evaluation*: For each scenario $s \in \mathscr{S}$, it solves two distinct problems: one in which the service to i^{k+1} is forced and one in which the service to i^{k+1} is forbidden. Let $z_{in}(s)$ and $z_{out}(s)$ be the value of the solutions obtained for scenario s when forcing or forbidding the service of i^{k+1}, respectively.
4. *Decision*: If $\sum_{s \in \mathscr{S}} z_{in}(s) \geqslant \sum_{s \in \mathscr{S}} z_{out}(s)$, then i^{k+1} is served otherwise the service is declined. This means that i^{k+1} is served if the approximated expected profit of accepting i^{k+1} is higher than the one of rejecting its service.

The two approaches that we propose are the following.

MC Greedy. In `MC Greedy` approach, Phase 3 (*Evaluation*) is implemented as follows: given all decisions taken up to time t^{k+1}, a greedy algorithm is used to decide on requests in scenario s. Thus, as we have four myopic greedy algorithms, we obtained four `MC Greedy` algorithms. We call each algorithm with the name of the corresponding myopic greedy algorithm preceded by `MC`.

MC OP-Multi-Shot. In `MC OP-Multi-Shot`, Phase 3 (Evaluation) is implemented as follows: given all decisions taken up to time t^{k+1}, a new deterministic static problem is generated using the events in scenario s. In particular, all request in W^k are set as mandatory with their respective prizes, request i^{k+1} is either mandatory or forbidden, according to which problem is solved, and requests in current scenario s are considered as deterministic requests with a prize equal to the expected prize (prize multiplied by the probability at time t^{k+1}). We solve the deterministic problem by adapting the heuristic algorithm presented in Chao et al. (1996) for the OP.

6. Computational tests

In this section, we present the computational tests that we made in order to verify the efficacy of the solution algorithms presented in Section 5 and to identify whether the performance of the different solution approaches depends on problem characteristics. We first describe the instances on which tests have been performed in Section 6.1 and then show and comment the results in Section 6.2. Tests are made on a Windows 64-bit machine equipped with Intel Xeon processor E5-1650, 3.50 GHz, and 16 GB Ram. All algorithms are coded in Java.

6.1. Test instances

As the DPOP has not been studied before in the literature, there are no benchmark instances to work with. We adapted the benchmark instances for the Probabilistic Orienteering Problem (POP) introduced in Angelelli et al. (2017) and modified them to take into account the characteristics of the DPOP and to study the impact of its peculiarities on solution algorithms. To ease readability, we start by describing the instances generated in Angelelli et al. (2017) for the POP.

Coordinates for customers and the depot are taken from TSP benchmark instances provided in the TSPLIB95 library available at the following url: http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95. We assume unit speed for the vehicle and $C = 1$, so that distance, time and cost coincide. We selected the subset of instances with a number of vertices ranging from 14 to 52, for a total of 16 graphs. In every instance, the first node is chosen as both the origin and the destination.

The remaining problem characteristics are set as follows:

- The value of $D_{\max}$ is chosen so that all the mandatory nodes and an increasing fraction of the optional nodes can be visited. We set $D_{\max} = \tau_{\min}(M) + \omega(\tau_{\min}(V) - \tau_{\min}(M))$ where $\tau_{\min}(S)$ is the duration (length) of the minimum cost elementary path from origin 0 to destination $n+1$ visiting all nodes in S. As done in Angelelli et al. (2017), we tested three values of ω: 1/4, 1/2 and 3/4.
- The percentage of mandatory customers with respect to the total number of customers in V has been set equal to two values: 0% and 25%.
- The values of prizes p_i for $i \in V$ has been generated according to four rules:

  P_1. All prizes are equal to 1.
  P_2. The prize of customer i is set to $1 + ((7141{\cdot}i + 73) \bmod 100)$ as done in Fischetti et al. (1998).
  P_3. The prize of customer i is set to $1 + \left\lceil \frac{99{\cdot}d_{0i}}{\max_{j \in V} d_{0j}} \right\rceil$ where d_{ij} is the distance between node i and node j.
  P_4. The prize of customer i is set to d_{0i}.

  To obtain a balanced trade-off between prizes and costs, in each instance the prizes are normalized as follows. First, they are scaled so that their sum is twice $C{\cdot}\tau_{\min}(V)$; second, each prize is rounded to the nearest integer.
- The probability of appearance of an optional request from a node $i \in V \backslash V_M$, i.e., θ_i, is generated in two ways:

– Constant for all customers. We tested three different situations: low probability 0.25 (in the following F_1), medium probability 0.50 (in the following F_2), high probability 0.75 (in the following F_3).
– Randomly generated in the interval $[0.25, 0.75]$ (in the following F_4).

Concerning the probability distributions, we assume that $\overline{T}_i = U[0, T]$ for all $i \in V \backslash V_M$, where $U[0, T]$ is the continuous uniform distribution taking values between 0 and T. In this way, the conditional probabilities $\theta_i^{k+1} = F_{i|t^{k+1}}(T)$ are easily computed as $\theta_i^{k+1} = \frac{\theta_i(T-t^{k+1})}{T-\theta_i t^{k+1}}$.

In summary, we have 96 parameter combination. For each TSPLIB instance and parameter combination, we generated 1000 random realizations. So, we tested in total 1,536 instances and 1000 scenarios for each of them. All instances are available at https://or-brescia.unibs.it/instances.

To guarantee a fair comparison of results, the scenarios used in the dynamic approaches have been generated once for each instance and used by all the approaches.

6.2. Computational results

In this section, we present the computational results obtained by the solution algorithms presented in Section 5. We first focus (Section 6.2.1) on the analysis of the greedy algorithms presented in Section 5.2.1 to compare the performance of the four rules we propose to determine the acceptance of a request. Then, in Section 6.2.2, we compare the myopic greedy algorithms with the `MC Greedy` ones to verify whether the introduction of the MC simulation improves the performance of the approaches. Section 6.2.3 show the results related to all approaches presented in Section 5. Finally, in Section 6.2.4 we report an analysis of the cost components and acceptance rate of the solutions provided by the different approaches.

The objective values returned by all algorithms are compared with those returned by a `Post-exact` algorithm, which provides the optimal solution of the problem using full "a posteriori" information about customer requests. More precisely, the `Post-exact` algorithm solves a POP over the set of (mandatory and optional) nodes arising requests in $(0, T)$, as this information is available at time T. The visit on mandatory nodes is imposed, while the probability of optional nodes is set to one. The value of the optimal solution of the resulting deterministic problem provides an upper bound on the value of the has not been studied before in the liter. The branch-and-cut algorithm proposed in Angelelli et al. (2017) for the POP has been used to determine the `Post-exact` solution.

6.2.1. Myopic greedy algorithms

We first focus on the analysis of the myopic greedy algorithms presented in Section 5.2.1. The reason is that greedy algorithms are based on simple rules and are likely to be applied in practical applications where no information is available on future events or when no sophisticated solution approach is available. Thus, it is relevant to determine whether there is a dominance between them and/or whether the performance depends on problem characteristics.

First, we analyze the average solution quality over all tested instances. The behavior is illustrated in Fig. 2, where the average solution value of the four greedy algorithms is illustrated and compared with the average solution value of the `Post-exact` algorithm.

The figure shows that `Profitable Look-ahead Greedy` and `Feasible Look-ahead Greedy` outperform both `Profitable Greedy` and `Feasible Greedy`, with the latter being slightly better than the former. In addition, despite being founded on basic rules, their gap with respect to the bound provided by `Post-exact` is reasonable.

A more detailed analysis of the behavior of myopic greedy algorithms on the basis of problem characteristics is provided in Table 1. The table reports, for each greedy algorithm, the percentage gap of the average solution value over all instances with respect to the average solution value obtained by `Post-exact`. The gap is calculated as:

$$\frac{avg(\texttt{myopicGreedy}) - avg(\texttt{Post-exact})}{avg(\texttt{Post-exact})}$$

where $avg(A)$ is the average solution value over all instances obtained by algorithm A and `myopic` refers to any one of the four algorithmic variants considered here. The results are classified by: values of ω, percentage of mandatory customers (called M from now on), rule for generating the probability of a request from an optional node and rule for generating customers' profits. The last line of the table reports the average over all instances.

The results confirm that the `Look-ahead Greedy` methods are significantly better than the `Greedy` ones. Since the advantage of `Feasible Look-ahead Greedy` with respect to `Profitable Look-ahead Greedy` is not remarkable, we applied the non-parametric Wilcoxon signed-rank test on paired data (Wilcoxon, 1945), testing the null hypotesis H_0 that `Feasible Look-ahead Greedy` and `Profitable Look-ahead Greedy` have different gap distributions. We obtained a p-value of 0.04785, slightly smaller than the standard significance level of 0.05. Consequently, we consider `Feasible Look-ahead Greedy` our best myopic greedy algorithm, with an average percentage gap slightly below 20%.

On the other hand, `Profitable Greedy` appears to be the worst algorithm, with an average percentage gap of almost 36%. The reason for the bad behavior of `Profitable Greedy` is related to the fact that it is based on a too conservative rule which prevents accepting requests that do not seem to be convenient immediately but may turn out to be convenient when combined with future requests. This is particularly evident when looking at the results classified by value of M: when $M = 0\%$, then the gap of `Profitable Greedy` is huge, almost 72%. In fact, in this case, as no request is mandatory, the cost related to the insertion of the first customer placing a request may turn out to be very high as the route is empty at the beginning, due to the absence of mandatory nodes. This induces `Profitable Greedy` to accept very few (if any) requests. When instead $M = 25\%$, the insertion cost may be lower and this favor the acceptance of requests. In addition, in the latter case, part of the solution, the one that serves the mandatory nodes, is in common with the solution of `Post-exact`, meaning that these customers are necessarily served by both solutions, and this reduces the gap. This trend is confirmed for all myopic greedy algorithms: the gap with respect to `Post-exact` is much larger for instances where $M = 0\%$ than for instances where $M = 25\%$ for the same reason explained above. Focusing on `Feasible Greedy`, we notice that it is much more sensitive with respect to `Profitable Greedy` to the value of ω, with the gap that increases when the value of ω decreases. This is explained by the fact that when the value of ω is small, then only few requests may be feasibly accepted, thus acceptance of early arrived requests prevents from serving future, possibly more profitable, ones. We also notice that `Feasible Greedy` is very much sensitive to the rules of generating the probability of customers' requests, providing a large gap (more than 50%) when all nodes have a probability of 50%. The reason may be the same mentioned above: when all nodes have the same probability of placing a request, an early acceptance may prevent the acceptance of (probable) convenient future requests. `Profitable Look-ahead Greedy` and `Feasible Look-ahead Greedy` show a very similar behavior. They always outperform the other two heuristics, with `Feasible Look-ahead Greedy` being dominant except for the cases where $\omega = 3/4, M = 0\%$ and F_2. Finally, we notice a quite remarkable variance of the results on the basis of the rules used to generate the profits. In particular, classes P_3 and P_4 are the ones for which all myopic greedy algorithms provide the worst results. This may be related to the fact that, when profits are related to distances, greedy rules based on the evaluation of functions that depend on the comparison between profits

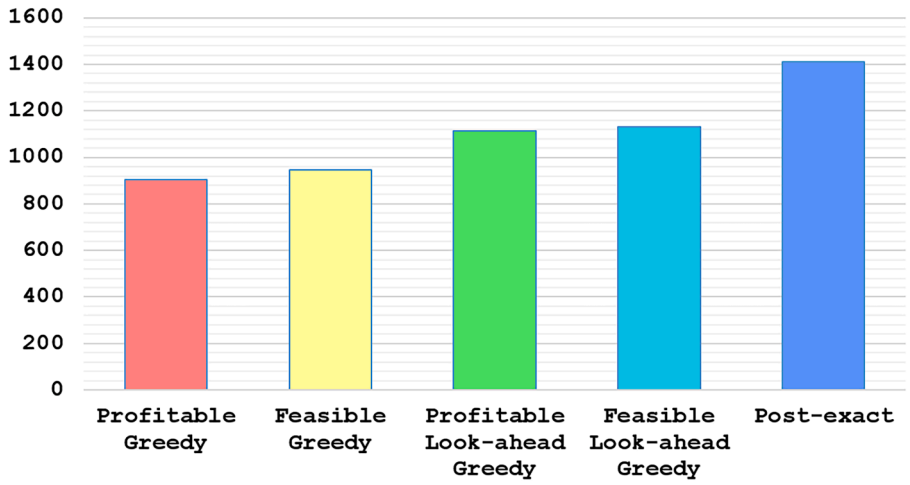

Fig. 2. Myopic greedy algorithms and `Post-exact`: average solution values.

Table 1
Performance of myopic greedy algorithms with respect to `Post-exact`.

	Profitable Greedy	Feasible Greedy	Profitable Look-ahead Greedy	Feasible Look-ahead Greedy
$\omega = 1/4$	33.54	45.83	29.25	23.06
$\omega = 1/2$	36.79	37.11	20.94	20.33
$\omega = 3/4$	36.40	20.56	15.64	17.11
$M = 0\%$	71.73	51.67	34.42	35.58
$M = 25\%$	12.33	20.69	12.24	9.45
F_1	34.04	28.02	22.24	21.25
F_2	21.52	50.18	19.80	19.96
F_3	34.30	28.79	23.46	23.02
F_4	40.52	35.10	19.03	16.93
P_1	34.55	29.23	18.62	18.61
P_2	27.21	36.56	15.49	13.34
P_3	41.79	32.70	25.70	24.37
P_4	41.74	32.84	25.69	24.35
All	**35.81**	**32.93**	**21.00**	**19.78**

and distances may face more troubles in establishing what is the best choice.

6.2.2. *Myopic versus MC greedy algorithms*

In this section, we analyze the benefits of introducing Monte Carlo simulation on the greedy algorithms evaluated above. The aim is to verify whether the introduction of Monte Carlo to simulate the evolution of future events helps in providing better quality solutions. Thus, we compare the performance of myopic versus MC greedy algorithms. In particular, for `MC Greedy`, we generated 100 scenarios for Step 2 of the procedure described in Section 5.2.2.

Table 2 shows the results. In particular, the table reports, for each `MC greedy` algorithm, the percentage gap with respect to the `Post-exact` solution and the percentage improvement with respect to the corresponding myopic greedy algorithm ('impr.'). Table 2 shows that the improvement due to the introduction of Monte Carlo simulation is high, especially when focusing on `Profitable Greedy` and `Feasible Greedy`. This can be safely attributed to the fact that these two heuristics are those which provide the worst results in their myopic version, and thus there is a much wider space for improvement in their case. Overall, MC provides big advantages to all greedy algorithms by remarkably smoothing the gaps with respect to `Post-exact`. In fact, focusing on `Profitable Look-ahead Greedy` and `Feasible Look-ahead Greedy`, we see that the gap is always lower than 12% except for one case ($\omega = 1/4$). Notice that, though the differences between `MC Profitable Look-ahead Greedy` and `MC Feasible Look-ahead Greedy` appear to be small, the Wilcoxon test run on the gap values returns $p = 0.00265$, denoting a statistically significant difference.

Table 3 shows the average computational time over all instances for all greedy approaches: the four myopic approaches and the four MC approaches with 100 scenarios. Times are reported in milliseconds (ms). The table shows that, even if the effort due to scenario evaluation is substantial, still computational times are absolutely acceptable. In fact, while they are at most 26 ms for myopic approaches, the highest average computational time for MC approaches is 305 ms for `MC Feasible Look-ahead Greedy` which is still extremely fast.

In Table 3, we note a counter-intuitive fact: while in the myopic version `Profitable Look-ahead Greedy` is slightly slower than `Feasible Look-ahead Greedy`, in the Monte Carlo version the former algorithm is much faster than the latter. We explain this phenomenon as follows. Under the same conditions, a request accepted by `Feasible Look-ahead Greedy` is accepted also by `Profitable Look-ahead Greedy`, but not vice versa. Hence, `Profitable Look-ahead Greedy` accept more requests at the beginning of the decision time window $(0, T)$ and might reduce more quickly the time budget for future requests. The smaller time budget imply that the simulations stop

Table 2
MC versus myopic greedy algorithms.

	MC Profitable Greedy		MC Feasible Greedy		MC Profitable Look-ahead Greedy		MC Feasible Look-ahead Greedy	
	gap	impr.	gap	impr.	gap	impr.	gap	impr.
$\omega = 1/4$	13.34	20.19	15.04	30.79	12.48	16.77	12.11	10.95
$\omega = 1/2$	9.34	27.45	9.60	27.51	8.21	12.73	7.63	12.70
$\omega = 3/4$	5.19	31.21	7.08	13.47	4.74	10.90	4.63	12.48
$M = 0\%$	12.87	58.85	15.24	36.43	10.87	23.55	10.65	24.93
$M = 25\%$	6.06	6.27	6.59	14.09	6.05	6.19	5.62	3.83
F_1	7.94	26.10	8.53	19.50	7.34	14.90	7.14	14.11
F_2	11.93	9.59	21.00	29.18	11.57	8.23	11.70	8.26
F_3	8.27	26.03	8.89	19.90	7.66	15.80	7.51	15.51
F_4	8.93	31.58	9.49	25.62	7.82	11.21	7.18	9.75
P_1	8.18	26.37	8.96	20.27	7.34	11.28	7.09	11.52
P_2	7.74	19.47	9.11	27.45	7.11	8.38	6.76	6.58
P_3	9.71	32.08	11.13	21.58	8.82	16.88	8.42	15.95
P_4	9.68	32.06	11.16	21.68	8.81	16.89	8.41	15.94
All	**8.76**	**27.05**	**10.01**	**22.92**	**7.96**	**13.05**	**7.61**	**12.17**

Table 3
Computational time of greedy algorithms (ms).

	Myopic	MC
Profitable Greedy	20	103
Feasible Greedy	26	110
Profitable Look-ahead Greedy	24	140
Feasible Look-ahead Greedy	21	305

earlier, as no simulation is run if a node cannot be feasibly inserted in the current path. This produces sensible time savings.

Given that Feasible Look-ahead Greedy provides a good compromise between solution quality and computational time and is the best greedy algorithm, in the following analysis we focus on it and do not provide further analysis related to the other greedy approaches.

One crucial decision in MC greedy approaches is the number of scenarios generated. Thus, we now analyse the performance of the MC Feasible Look-ahead Greedy on the basis of this parameter. In Fig. 3, we report the average solution value of the Feasible Look-ahead Greedy, MC Feasible Look-ahead Greedy with 10, 50, and 100 scenarios and the Post-exact solution. What is interesting to notice is that the advantage to the MC Feasible Look-ahead Greedy with respect to Feasible Look-ahead Greedy is achieved already with 10 scenarios.

We remark that a similar analysis has been performed also on the other greedy algorithms and the results showed a similar trend.

6.2.3. *Comparison of static and dynamic approaches*

In this section, we analyze the performance of all approaches illustrated in Section 5. As mentioned above, from the class of greedy algorithms, we choose the Feasible Look-ahead Greedy as a good compromise between solution quality and computational time. Thus, in the following we illustrate the results related to the following solution approaches:

- OP-One-Shot.
- OP-Multi-Shot.
- Feasible Look-ahead Greedy.
- MC Feasible Look-ahead Greedy with 100 scenarios. Even if we noticed in the previous section that the results with 10 scenarios are similar to the ones with 100 scenarios, given the extremely low computation time of MC Feasible Look-ahead Greedy with 100 scenarios, we kept this setting as it is the one that provides the best results.

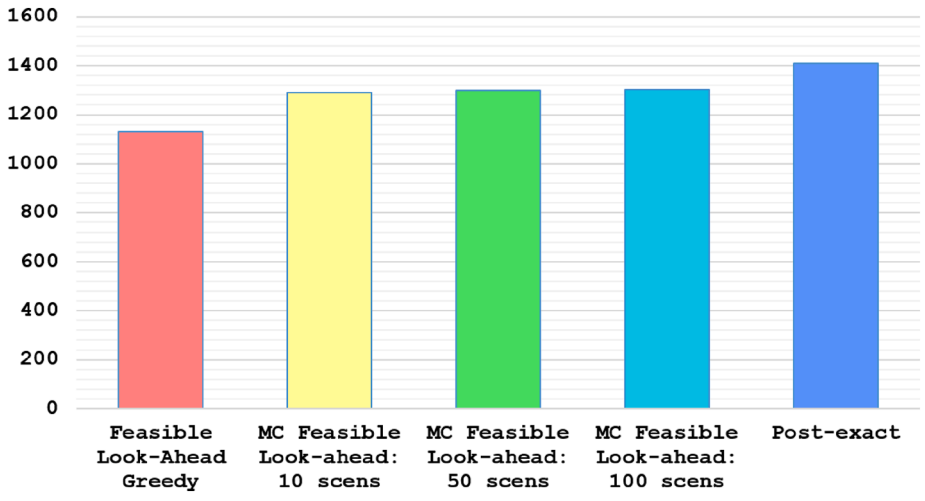

Fig. 3. Comparison of Feasible Look-ahead Greedy and MC Feasible Look-ahead Greedy with different number of scenarios (scens).

- MC OP-Multi-Shot with 10 scenarios. In this case, we choose a number of scenarios equal to 10 as MC OP-Multi-Shot is much more time consuming than MC Feasible Look-ahead Greedy. We will analyse more in depth this issue in the following.

First, we focus on the comparison between MC OP-Multi-Shot and MC Feasible Look-ahead Greedy. In particular, we analyze the average solution quality to determine which approach is performing better. In Fig. 4, we show the average solution value, over all instances, of MC OP-Multi-Shot and MC Feasible Look-ahead Greedy with 10 and 100 scenarios respectively. The figure shows that the best approach among the three is MC Feasible Look-ahead Greedy with 100 scenarios, but the difference is very limited. In addition, MC Feasible Look-ahead Greedy with 10 scenarios performs better than MC OP-Multi-Shot, which has the same number of scenarios. This may be due to the fact that, even if MC feasible Look-ahead Greedy is based on a simple decision rule, the advantage of taking into account the time at which events occur overcomes the more sophisticated decision rule applied in MC OP-Multi-Shot.

Extending the comparison to the remaining approaches listed above, we obtain the results presented in Table 4. The table reports the gap with respect to the solution of the Post-exact algorithm categorized as done in the previous tables.

Table 4 shows that the best approach is MC Feasible Look-ahead Greedy which systematically outperforms all other approaches on all instance classes. The worst approach is OP-One-Shot. This was expected as OP-One-Shot is a static approach which determines a solution at time 0 and decisions are based on this solution without taking into account updated information. In addition, OP-One-Shot behaves particularly bad for the class of instances F_2, where all optional nodes have a probability equal to 0.5 of placing a request. This is the class of instances that generates the worst performance also for the other approaches, and this is due to the fact that this case generates the highest error in predicting which customers will place a request. However, this disadvantage is amplified in OP-One-Shot due to its static nature. Concerning the comparison between OP-Multi-Shot and MC OP-Multi-Shot, we see that the latter performs better with the exception of few cases, i.e., $\omega = 1/4, M = 25\%$ and F_4.

Finally, in Table 5 we present the average computational times (in ms) of the approaches analyzed in Table 4, with the same classification of instances.

All computational times are absolutely reasonable being at most slightly above half a second. OP-One-Shot has a computational time of few ms. However, as it is shown in Table 4, this goes to the detriment of solution quality. Among the three other approaches, the fastest is MC Feasible Look-ahead Greedy which confirms to be the best both in

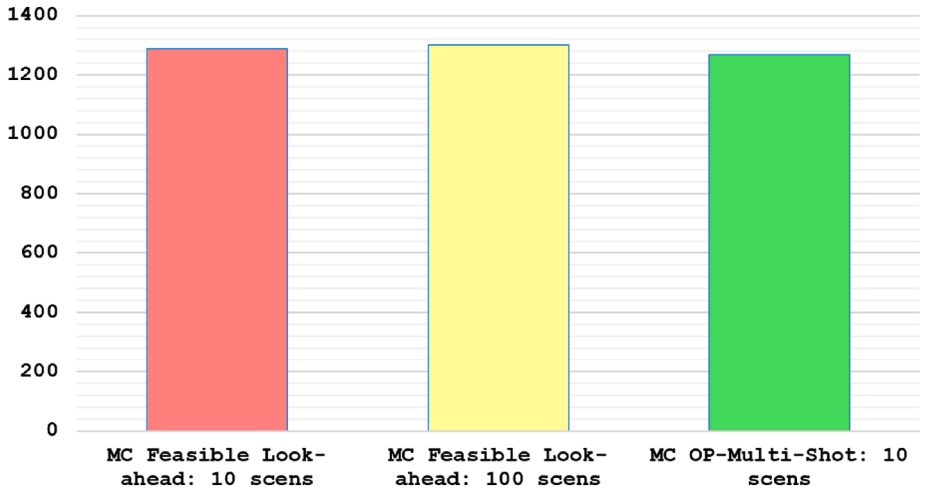

Fig. 4. Comparison of MC Feasible Look-ahead Greedy and MC OP-Multi-Shot.

Table 4
Performance of static and dynamic approaches with respect to Post-exact.

	OP-One-Shot	OP-Multi-Shot	MC Feasible Look-ahead Greedy	MC OP-Multi-Shot
$\omega = 1/4$	23.45	15.20	12.11	16.51
$\omega = 1/2$	21.87	14.89	7.63	10.07
$\omega = 3/4$	20.31	13.00	4.63	5.69
$M = 0\%$	32.83	25.74	10.65	13.47
$M = 25\%$	14.38	6.72	5.62	7.76
F_1	25.76	17.91	7.14	8.79
F_2	60.98	20.57	11.70	14.46
F_3	27.30	19.03	7.51	8.99
F_4	8.34	7.95	7.18	10.54
P_1	20.03	12.02	7.09	9.33
P_2	18.26	10.69	6.76	9.05
P_3	24.61	17.64	8.42	11.00
P_4	24.72	17.63	8.41	10.98
All	**21.67**	**14.24**	**7.61**	**10.02**

Table 5
Computational times (ms).

	OP-One-Shot	OP-Multi-Shot	MC Feasible Look-ahead Greedy	MC OP-Multi-Shot
$\omega = 1/4$	8	302	140	278
$\omega = 1/2$	14	503	332	680
$\omega = 3/4$	19	437	445	579
$M = 0\%$	5	291	297	663
$M = 25\%$	23	537	314	361
F_1	12	359	305	280
F_2	4	112	76	28
F_3	11	352	297	273
F_4	28	832	544	1467
P_1	16	506	321	567
P_2	14	429	305	486
P_3	13	362	298	500
P_4	13	359	298	496
All	**14**	**414**	**305**	**512**

terms of solution quality and computational time.

6.2.4. Solutions analysis

In this section we, provide an analysis of the cost components and the acceptance rate of the solutions provided by the different approaches we proposed. In particular, we compare the following approaches:

- `OP-One-Shot`,
- `OP-Multi-Shot`,
- `MC OP-Multi-Shot`, with 10 scenarios,
- `Feasible Look-Ahead Greedy`,
- `MC Feasible Look-Ahead Greedy` with 100 scenarios.

Table 6 reports, for the five algorithms mentioned above, the average objective function value, the average value of the prize collected and the average value of the route cost. The latter is reported both in terms of absolute value and in terms of percentage with respect to the average value of the prize collected. Results are classified as in previous tables. The results show that the ratio between prize collected and solution cost follow very similar patterns for all algorithms. Thus, the different performance of the algorithms can be fully attributed to the ability of including a larger number of customers in the final solution. More in detail, we see that the ratio between prize collected and solution cost remains in the range from 60 to 65% for all approaches and all parameters, with the exception of the parameter related to the probability of appearance of optional requests. In fact, in the latter case, the ratio decreases to a minimum of 51% for F_4 and increases to a maximum of 88% for F_2. The steep increase related to setting F_2 can be explained by the fact that this is the most uncertain setting that generates the worst solutions, as witnessed by the average solution values of all algorithms.

Table 7 reports the acceptance rate of each algorithm, still classified as before. The acceptance rate is the percentage of optional requests served with respect to the requests placed (as sum of optional and mandatory requests). The algorithms with the maximum acceptance rate are `MC OP-Multi-Shot` and `MC Feasible Look-Ahead Greedy`, which are also the ones providing the best solutions (cf. Table 4). This confirms that there exists a positive correlation between solution quality and acceptance rate. When analyzing how the acceptance varies according to problem parameters, we see that it increases for instances where the number of mandatory customers is 25%, for obvious reasons. Concerning the probability of appearance of optional requests, we still have a steep decrease for F_2, confirming the difficulty of handling a highly uncertain setting. Finally, when looking at customers' prize, we see that the highest acceptance rate is related to the case where all prizes are equal to 1. This might be explained by the fact that, in this setting, the convenience of accepting a customer request solely depends on the detour cost for serving the customer. Thus, if this cost is small enough, it is convenient to accept the customer request as future requests will not provide a larger prize with respect to the current request.

7. Conclusions

We have studied a dynamic and probabilistic OP, called DPOP, where decisions about acceptance/rejection of nodes must be taken one at a time, according to the probabilistic appearance of new requests. Given the set of assumptions, we have derived explicit formulas for the optimal policy according to the objective of expected profit maximization. We have designed and implemented different heuristic techniques

Table 6
Analysis of components of solution value.

	OP-One-Shot			OP-Multi-Shot			MC OP-Multi-Shot			Feasible Look-ahead Greedy			MC Feasible Look-ahead Greedy		
	obj.	prize	cost	obj.	prize	cost	obj.	prize	cost	obj.	prize	cost	obj.	prize	cost
$\omega = 1$	823	2330	1507 – 65%	912	2511	1599 – 64%	898	2531	1633 – 65%	827	2342	1514 – 65%	945	2608	1663 – 64%
$\omega = 2$	1187	2966	1779 – 60%	1294	3235	1941 – 60%	1367	3527	2160 – 61%	1211	3032	1821 – 60%	1404	3583	2179 – 61%
$\omega = 3$	1301	3213	1911 – 59%	1421	3525	2104 – 60%	1540	3937	2397 – 61%	1354	3348	1994 – 60%	1558	3980	2423 – 61%
$M = 0\%$	748	1485	737 – 50%	827	1724	897 – 52%	964	2216	1251 – 56%	718	1443	725 – 50%	996	2319	1324 – 57%
$M = 25\%$	1460	4188	2728 – 65%	1590	4457	2867 – 64%	1572	4448	2876 – 65%	1544	4371	2828 – 65%	1609	4462	2853 – 64%
F_1	1072	2793	1721 – 62%	1186	3076	1890 – 61%	1318	3488	2170 – 62%	1138	2914	1777 – 61%	1341	3529	2188 – 62%
F_2	175	1422	1246 – 88%	357	1771	1414 – 80%	385	1923	1539 – 80%	360	1773	1413 – 80%	397	2029	1632 – 80%
F_3	1010	2699	1689 – 63%	1125	2989	1864 – 62%	1264	3417	2153 – 63%	1069	2807	1738 – 62%	1285	3464	2179 – 63%
F_4	2158	4432	2273 – 51%	2168	4525	2358 – 52%	2107	4499	2392 – 53%	1956	4134	2178 – 53%	2186	4540	2355 – 52%
P_1	1154	2947	1792 – 61%	1270	3244	1974 – 61%	1309	3396	2087 – 61%	1175	3028	1853 – 61%	1341	3456	2114 – 61%
P_2	1301	3106	1805 – 58%	1421	3397	1976 – 58%	1448	3507	2060 – 59%	1379	3313	1933 – 58%	1484	3556	2072 – 58%
P_3	981	2647	1666 – 63%	1072	2860	1788 – 63%	1158	3212	2054 – 64%	985	2645	1660 – 63%	1192	3276	2084 – 64%
P_4	980	2645	1666 – 63%	1072	2860	1788 – 63%	1158	3212	2053 – 64%	984	2644	1659 – 63%	1192	3275	2083 – 64%
All	**1104**	**2836**	**1732 – 61%**	**1209**	**3090**	**1882 – 61%**	**1268**	**3332**	**2063 – 62%**	**1131**	**2907**	**1777 – 61%**	**1302**	**3390**	**2088 – 62%**

Table 7
Acceptance rate.

	OP-One-Shot	OP-Multi-Shot	MC OP-Multi-Shot	Feasible Look-ahead Greedy	MC Feasible Look-ahead Greedy
$\omega = 1$	0.47	0.53	0.54	0.49	0.56
$\omega = 2$	0.56	0.64	0.71	0.58	0.73
$\omega = 3$	0.61	0.69	0.79	0.64	0.81
$M = 0\%$	0.29	0.36	0.49	0.29	0.54
$M = 25\%$	0.80	0.87	0.87	0.86	0.87
F_1	0.57	0.64	0.73	0.58	0.74
F_2	0.40	0.52	0.58	0.52	0.63
F_3	0.54	0.61	0.72	0.56	0.74
F_4	0.67	0.70	0.69	0.62	0.70
P_1	0.60	0.69	0.73	0.64	0.75
P_2	0.59	0.67	0.69	0.64	0.71
P_3	0.50	0.55	0.65	0.50	0.68
P_4	0.50	0.55	0.65	0.50	0.67
All	**0.54**	**0.62**	**0.68**	**0.57**	**0.70**

according to three approaches: static (i.e., using a fixed, a priori view of the future evolution), myopic (i.e., without anticipation of future events), anticipatory (i.e., based on SAA of the objective function through Monte Carlo simulation of future events). Computational tests have shown that simple anticipatory approaches are superior to myopic and static approaches.

The special feature of DPOP is the fact that acceptance/rejection decisions have to be made in real time but before the vehicle starts its route. Hence, the set of accepted nodes and the whole sequence of visits must be updated each time a new request is accepted.

The present work may be extended along at least two directions. First, a richer environment may be considered, including multiple vehicles, capacity constraints, and customer time windows. This would not impact the decision framework, but it would force the adoption of more sophisticated algorithmic approaches. Second, a different management of the decision time interval $(0, T)$ could be considered. To model an online decision process, we have considered time as a continuous parameter. In many applications however, time is considered as a discrete parameter. If time is discretized in equally spaced decision epochs, multiple simultaneous service requests must be taken into account. Thus, considering time as a discrete parameter would impact both the theoretical modeling and the algorithmic approaches.

CRediT authorship contribution statement

Enrico Angelelli: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization. **Claudia Archetti:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization. **Carlo Filippi:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization. **Michele Vindigni:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization.

Appendix A. Proof of Theorem 1

We start introducing some notations. For all $i \in V_S$ and all $t^\star \in (0, T)$, let $F_{i|t^\star}$ denote the CDF of the random variable T_i, given that no request has occurred from i in $[0, t^\star]$. For all $t \in (t^\star, T)$, we have:

$$F_{i|t^\star}(t) = \frac{\theta_i(\overline{F}_i(t) - \overline{F}_i(t^\star))}{1 - \theta_i \overline{F}_i(t^\star)} \tag{7}$$

(see (5)). Moreover, we have:

$$F_{i|t^\star}(t) = \begin{cases} 0 & \text{if } t \leqslant t^\star, \\ \dfrac{\theta_i - F_i(t^\star)}{1 - F_i(t^\star)} = \dfrac{1 - \overline{F}_i(t^\star)}{1 \Big/ \theta_i - \overline{F}_i(t^\star)} & \text{if } T \leqslant t < T + 1, \\ 1 & \text{if } t \geqslant T + 1. \end{cases}$$

Recall that $\bar{f}_i$ denotes the PDF of $\overline{T}_i$. Accordingly, let $\bar{f}_{i|t^\star}$ denote the PDF of $\overline{T}_i$, given that no request has occurred from i in $[0, t^\star]$. By deriving (7) we have:

$$\bar{f}_{i|t^\star}(t) = \frac{\theta_i}{1 - \theta_i \overline{F}_i(t^\star)} \bar{f}_i(t) \qquad \text{for all } t \in (t^\star, T) \tag{8}$$

(See Fig. 5 for an illlustration.)

To prove the theorem, assume that the proper stage $(t^{k+1}, \bar{t}^{k+1})$ occurs with system at state (R^k, W^k). First, we prove that Eq. (3) defines the

probability that the next stage will be proper and will involve node $i \in V \backslash R^{k+1}$.

Let $i \in V \backslash R^{k+1}$ and let $t \in (t^{k+1}, T)$. From the independence assumption and using (7), the probability that $T_j > t$ for all $j \in V \backslash (R^{k+1} \cup \{i\})$ is

$$\prod_{j \in V \backslash (R^{k+1} \cup \{i\})} \left(1 - F_{j|t^{k+1}}(t)\right) = \prod_{j \in V \backslash (R^{k+1} \cup \{i\})} \frac{1 - \theta_i \overline{F}_j(t)}{1 - \theta_i \overline{F}_j(t^{k+1})}.$$

From the law of total probability, the next stage will be proper and involve $i \in V \backslash R^{k+1}$ with probability

$$\int_{t^{k+1}}^{T} \left[\prod_{j \in V \backslash (R^{k+1} \cup \{i\})} \frac{1 - \theta_i \overline{F}_j(t)}{1 - \theta_i \overline{F}_j(t^{k+1})} \right] \overline{f}_{i|t^\star}(t) dt.$$

Thus, Eq. (3) follows from (8).

Second, we prove that (4) defines the probability that the next stage will be the final stage. The next stage is the final stage if $t_i = T+1$ for all $i \in V \backslash R^{k+1}$. Since

$$\mathbb{P}(t_i = T+1 | t_i > t^{k+1}) = 1 - F_{j|t^{k+1}}(T),$$

we have that Eq. (4) follows from the independence assumption.

Finally, we prove that the probabilities computed as in (3) and (4) form indeed a probability distribution, i.e.,

$$\sum_{i \in V \backslash R^{k+1}} p_{\min}(i | t^{k+1}, R^{k+1}) + p_{\min}(\varnothing | t^{k+1}, R^{k+1}) = 1 \tag{9}$$

By using the conditioned distributions, we may write:

$$p_{\min}(i | t^{k+1}, R^{k+1}) = \int_{t^{k+1}}^{T} \left[\prod_{j \in V \backslash (R^{k+1} \cup \{i\})} \left(1 - F_{j|t^{k+1}}(t)\right) \right] \overline{f}_{i|t^{k+1}}(t) dt \tag{10}$$

$$p_{\min}(\varnothing | t^{k+1}, R^{k+1}) = \prod_{j \in V \backslash R^{k+1}} \left(1 - F_{j|t^{k+1}}(T)\right)$$

For convenience, for all $i \in V \backslash R^{k+1}$, we consider the mixed random variables $\widehat{T}_i = T - T_i$. Given a $t^\star \in (0, T)$, for all $t \in [t^\star, T]$ we define:

$$\widehat{F}_{i|T-t^\star}(T-t) = \mathbb{P}(\widehat{T}_i \leqslant T - t | \widehat{T}_i < T - t^\star).$$

We have:

$$\begin{aligned} \widehat{F}_{i|T-t^\star}(T-t) &= \mathbb{P}(T - T_i \leqslant T - t | T - T_i < T - t^\star) \\ &= \mathbb{P}(T_i \geqslant t | T_i > t^\star) \\ &= 1 - \mathbb{P}(T_i \leqslant t | T_i > t^\star) = 1 - F_{i|t^\star}(t) \end{aligned}$$

Moreover, for all $t \in (t^\star, T)$,

$$\widehat{f}_{i|T-t^\star}(T-t) = \frac{d}{d(T-t)} \widehat{F}_{i|T-t^\star}(T-t) = -\frac{d}{dt}(1 - F_{i|t^\star}(t)) = \overline{f}_{i|t^\star}(t)$$

(See Fig. 6 for an illustration.) Thus, Eq. (10) can be written

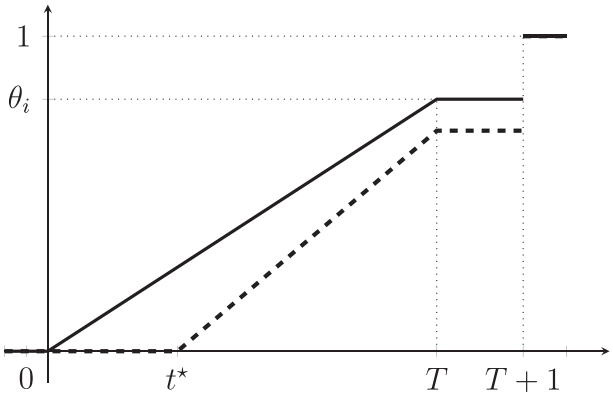

Fig. 5. Example plot of F_i (solid line) and $F_{i|t^\star}$ (dashed line) in the special case where $\overline{T}_i$ is uniform.

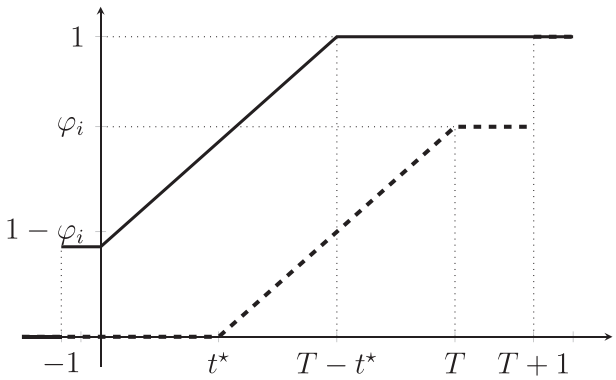

Fig. 6. Example plot of $\widehat{F}_{i|T-t^\star}$ (solid line) in comparison with $F_{i|t^\star}$ (dashed line) in the special case where $\overline{T}_i$ is uniform. We use φ_i to denote $F_{i|t^\star}(T)$.

$$
\begin{aligned}
p_{\min}(i|t^{k+1},R^{k+1}) &= \int_{t^{k+1}}^{T}\left[\prod_{j\in V\setminus(R^{k+1}\cup\{i\})}\left(\widehat{F}_{j|T-t^{k+1}}(T-t)\right)\right]\widehat{f}_{i|T-t^{k+1}}(T-t)dt\\
&= \int_{T}^{t^{k+1}}\left[\prod_{j\in V\setminus(R^{k+1}\cup\{i\})}\left(\widehat{F}_{j|T-t^{k+1}}(T-t)\right)\right]\widehat{f}_{i|T-t^{k+1}}(T-t)d(T-t)\\
&= \int_{0}^{T-t^{k+1}}\left[\prod_{j\in V\setminus(R^{k+1}\cup\{i\})}\left(\widehat{F}_{j|T-t^{k+1}}(x)\right)\right]\widehat{f}_{i|T-t^{k+1}}(x)dx
\end{aligned}
$$

Applying integration by parts to the last expression, we obtain:

$$
\begin{aligned}
p_{\min}(i|t^{k+1},R^{k+1}) &= \left[\prod_{j\in V\setminus R^{k+1}}\widehat{F}_{j|T-t^{k+1}}(x)\right]_0^{T-t^{k+1}} - \sum_{h\in V\setminus(R^{k+1}\cup\{i\})}\int_0^{T-t^{k+1}}\left[\prod_{j\in V\setminus(R^{k+1}\cup\{h\})}\left(\widehat{F}_{j|T-t^{k+1}}(x)\right)\right]\widehat{f}_{h|T-t^{k+1}}(x)dx = 1-\prod_{j\in V\setminus R^{k+1}}\left(1-F_{j|t^{k+1}}(T)\right)\\
-\sum_{h\in V\setminus(R^{k+1}\cup\{i\})}p_{\min}(h|t^{k+1},R^{k+1}) &= 1-p_{\min}(\varnothing|t^{k+1},R^{k+1})-\sum_{h\in V\setminus(R^{k+1}\cup\{i\})}p_{\min}(h|t^{k+1},R^{k+1})
\end{aligned}
$$

This proves Eq. (9) and completes the proof.

References

Agatz, N., Campbell, A., Fleischmann, M., Savelsbergh, M., 2011. Time slot management in attended home delivery. Transportation Science 45 (3), 435–449.

Allahviranloo, M., Chow, J.Y.J., Recker, W.W., 2014. Selective vehicle routing problems under uncertainty without recourse. Transportation Research Part E: Logistics and Transportation Review 62, 68–88.

Angelelli, E., Archetti, C., Filippi, C., Vindigni, M., 2017. The probabilistic orienteering problem. Computers & Operations Research 81, 269–281.

Azi, N., Gendreau, M., Potvin, J.-Y., 2012. A dynamic vehicle routing problem with multiple delivery routes. Annals of Operations Research 199 (1), 103–112.

Bent, R.W., Van Hentenryck, P., 2004. Scenario-based planning for partially dynamic vehicle routing with stochastic customers. Operations Research 52 (6), 977–987.

Campbell, A.M., Savelsbergh, M.W.P., 2005. Decision support for consumer direct grocery initiatives. Transportation Science 39 (3), 313–327.

Campbell, A.M., Savelsbergh, M., 2006. Incentive schemes for attended home delivery services. Transportation Science 40 (3), 327–341.

Campbell, A.M., Thomas, B.W., 2008. Probabilistic traveling salesman problem with deadlines. Transportation Science 42 (1), 1–21.

Campbell, A.M., Gendreau, M., Thomas, B.W., 2011. The orienteering problem with stochastic travel and service times. Annals of Operations Research 186 (1), 61–81.

Chao, I.-M., Golden, B.L., Wasil, E.A., 1996. A fast and effective heuristic for the orienteering problem. European Journal of Operational Research 88 (3), 475–489.

Evers, L., Glorie, K., van der Ster, S., Barros, A.I., Monsuur, H., 2014. A two-stage approach to the orienteering problem with stochastic weights. Computers & Operations Research 43, 248–260.

Fischetti, M., Salazar González, J.J., Toth, P., 1998. Solving the orienteering problem through branch-and-cut. INFORMS Journal on Computing 10 (2), 133–148.

Flatberg, T., Hasle, G., Kloster, O., Nilssen, E.J., Riise, A., 2007. Dynamic and stochastic vehicle routing in practice. In: Zeimpekis, V., Tarantilis, C.D., Giaglis, G.M., Minis, I. (Eds.), Dynamic Fleet Management: Concepts, Systems, Algorithms & Case Studies. Springer US, Boston, MA, pp. 41–63.

Ghiani, G., Manni, E., Quaranta, A., Triki, C., 2009. Anticipatory algorithms for same-day courier dispatching. Transportation Research Part E: Logistics and Transportation Review 45 (1), 96–106.

Ghiani, G., Manni, E., Thomas, B.W., 2012. A comparison of anticipatory algorithms for the dynamic and stochastic traveling salesman problem. Transportation Science 46 (3), 374–387.

Gunawan, A., Lau, H.C., Vansteenwegen, P., 2016. Orienteering problem: A survey of recent variants, solution approaches and applications. European Journal of Operational Research 255 (2), 315–332.

Hvattum, L.M., Løkketangen, A., Laporte, G., 2006. Solving a dynamic and stochastic vehicle routing problem with a sample scenario hedging heuristic. Transportation Science 40 (4), 421–438.

İlhan, T., Iravani, S.M.R., Daskin, M.S., 2008. The orienteering problem with stochastic profits. IIE Transactions 40 (4), 406–421.

Pillac, V., Gendreau, M., Guéret, C., Medaglia, A.L., 2013. A review of dynamic vehicle routing problems. European Journal of Operational Research 225 (1), 1–11.

Powell, W.B., Simao, H.P., Bouzaiene-Ayari, B., 2012. Approximate dynamic programming in transportation and logistics: a unified framework. EURO Journal on Transportation and Logistics 1, 237–284.

Psaraftis, H.N., Wen, M., Kontovas, C.A., 2016. Dynamic vehicle routing problems: Three decades and counting. Networks 67 (1), 3–31.

Ritzinger, U., Puchinger, J., Hartl, R.F., 2016. A survey on dynamic and stochastic vehicle routing problems. International Journal of Production Research 54 (1), 215–231.

Sungur, I., Ren, Y., Ordó nez, F., Dessouky, M., Zhong, H., 2010. A model and algorithm for the courier delivery problem with uncertainty. Transportation Science 44 (2), 193–205.

Ulmer, M.W., Thomas, B.W., 2019. Meso-parametric value function approximation for dynamic customer acceptances in delivery routing. European Journal of Operational Research.

Ulmer, M.W., Mattfeld, D.C., Köster, F., 2018. Budgeting time for dynamic vehicle routing with stochastic customer requests. Transportation Science 52 (1), 20–37.

Wilcoxon, F., 1945. Individual comparisons by ranking methods. Biometrics Bulletin 1 (6), 80–83.

Zhang, S., Ohlmann, J.W., Thomas, B.W., 2014. A priori orienteering with time windows and stochastic wait times at customers. European Journal of Operational Research 239 (1), 70–79.

Zhang, S., Ohlmann, J.W., Thomas, B.W., 2018. Dynamic orienteering on a network of queues. Transportation Science 52 (3), 691–706.